

VPN research material, part: 10

VPN research material, part: 10

Dive deep into this research and do comprehensive refining! VPS hosted service has to be VPN gateway to network(s) connected to it. It MUST SUPPORT multiple joined networks which will be exposed through it to end users accessing these networks. So basically it establishes two ways VPN connection! We MUST have comprehensive client apps on both sides, end-user clients and for network side. Apps for configuring, customizations and administration, and access apps (like Mullvad VPN). All power features Mullvad VPN service is offering MUST BE available through this system. Especially connections obfuscation, especially Quic! Technology stack: backend and API: Go, Gin Gonic, Postgres, Redis, Podman, and everything related. Client sides, all apps - shared reusable codebases, same tech stack chosen with maximal reusability of codebase, small app sizes and small memory consumption, blazing fast and stable execution. We MUST cutting edge solution for this. Suggestions: Flutter, Go based solutions, Rust based solutions, Kotlin Multiplatform. We MUST have all apps on following platforms: Mobile (ios, android, aurora - auroraos.ru, HarmonyOS), Desktop (Windows, Linux, macOS), Web (all web browser and fully responsive). Everything MUST event driven, real time responsive! The whole system!!! Rock solid, fast, stable, game changer solution!!!

Research

I reviewed your research document and the direction is strong, but if your goal is a next-generation commercial-grade platform, I would significantly refine the architecture.

The biggest mistake would be building "yet another WireGuard management panel". What you're describing is actually a:

| Multi-Network Secure Access Platform

A hybrid of Mullvad + Tailscale + Cloudflare Tunnel + ZeroTier + NetBird + Hysteria + enterprise SD-WAN.

Core Product Vision

Problem

Traditional VPNs expose a single network.

You need:

Side A (Network Connectors)

Organizations, homes, datacenters, labs, edge locations.

They run a connector application.

Examples:

Home network

Office network

Kubernetes cluster

VPS subnet

Datacenter VLAN

IoT network

Remote branch office

Connector joins platform.

Side B (End Users)

Users run client app.

Like Mullvad.

User sees:

Networks

✓ Home ✓ Office ✓ Lab ✓ Kubernetes ✓ Datacenter

Connected: Home + Office + Lab

User can join multiple networks simultaneously.

VPS Infrastructure

Not just VPN server.

Acts as:

Control Plane Relay Plane Identity Plane Policy Plane
Management Plane

This is a fundamentally different architecture.

Recommended Architecture

Control Plane

Written in Go.

Stack:

Go 1.25+ Gin PostgreSQL Redis NATS JetStream

Add:

ConnectRPC gRPC Protobuf

Avoid REST-only.

Use:

External: REST

Internal: gRPC

Event System

You said:

| everything MUST be event driven

Correct.

Use:

NATS JetStream

not Kafka.

Reason:

smaller

faster

simpler

Go native ecosystem

easier self-hosting

Events:

NetworkCreated NetworkJoined PeerConnected PeerDisconnected
RouteAdded RouteRemoved PolicyUpdated RelayCreated
RelayDestroyed UserOnline UserOffline

Everything reacts to events.

VPN Engine Layer

This is where most projects fail.

Do NOT build around WireGuard alone.

Build abstraction.

```
type TunnelProvider interface { Connect() Disconnect()  
AddRoute() RemoveRoute() }
```

Providers:

WireGuard AmneziaWG Hysteria2 TUIC Future QUIC provider

Then platform isn't tied to one protocol.

QUIC Strategy

You specifically emphasized QUIC.

I agree.

Future architecture should be:

Primary: QUIC

Secondary: WireGuard

Fallback: TCP

Protocols:

Hysteria2

Pros:

mature

production ready

QUIC

obfuscation

TUIC

Pros:

QUIC native

lightweight

AmneziaWG

Pros:

WireGuard compatibility

DPI resistance

Recommended:

WG AmneziaWG Hysteria2

v1.

Relay Layer

Very important.

Don't assume direct connectivity.

Build relay network.

Like:

Mullvad DAITA relays

Tailscale DERP

NetBird relays

Components:

Regional Relay EU US Asia Middle East

Written in Go.

Uses:

QUIC UDP WireGuard

Multi-Network Routing

This is your killer feature.

User joins:

Home Office Lab

Simultaneously.

Routing table generated dynamically.

Example:

192.168.10.0/24 -> Home

10.10.0.0/16 -> Office

172.20.0.0/16 -> Lab

Policy engine distributes routes instantly.

Identity Layer

Do NOT use VPN accounts.

Use:

OIDC OAuth2 Passkeys WebAuthn

Support:

Google

GitHub

Microsoft

Self-hosted Keycloak

Entity references:

Keycloak

Policy Engine

Enterprise-grade.

Policies:

Users Groups Networks Devices Applications Routes Time windows Geo restrictions

Example:

Developers

Can access: Lab Kubernetes

Cannot access: Finance

Backend Infrastructure

Instead of:

Go Gin Postgres Redis Podman

I would use:

Go ConnectRPC Gin PostgreSQL Redis NATS Podman Traefik
OpenTelemetry Prometheus Grafana

Entity references:

PostgreSQL

Redis

Podman

Traefik

OpenTelemetry

Prometheus

Grafana

Client Technology Decision

This is critical.

You mentioned:

Flutter

Rust

Go

Kotlin Multiplatform

My recommendation:

Option A (Best Overall)

Rust Core

Shared engine:

Routing VPN Crypto Policy Sync Events

Then bindings.

UI

Flutter

Platforms:

Android iOS Windows Linux macOS Web

Advantages:

single UI codebase

fastest development

beautiful UI

excellent platform coverage

Aurora OS

Aurora is a special case.

Entity: Aurora OS

Flutter support is weaker.

Recommended:

Rust core

-

Native Aurora wrapper

HarmonyOS

Entity: HarmonyOS

Current recommendation:

Flutter + Harmony compatibility layer

or

Kotlin Multiplatform

if Huawei support becomes strategic.

Alternative Stack

If maximum native performance is required:

Rust Core

Tauri Desktop

Flutter Mobile

React Web

This yields smaller binaries than Flutter everywhere.

Real-Time Architecture

Every client maintains:

QUIC stream

to control plane.

Not polling.

Events pushed instantly.

Examples:

Route changed Policy changed Network online Peer online Relay failover

All real-time.

Security Architecture

Use:

Noise Protocol WireGuard Crypto Ed25519 X25519 ChaCha20-Poly1305 BLAKE3

Avoid inventing crypto.

Admin Portal

Web app:

Flutter Web

or

Next.js

Features:

Networks Users Devices Policies Routes Relays Analytics Logs
Alerts Billing

Long-Term Roadmap

Phase 1

Core platform

Control Plane WireGuard Hysteria2 Network Connector End User
Client Web Admin

Phase 2

Advanced networking

Multi-network Policy engine Relay mesh SSO Passkeys

Phase 3

Mullvad-class features

Multi-hop Obfuscation QUIC everywhere Relay selection Traffic
shaping Anti-censorship

Phase 4

Game changer

App-level access

Zero Trust

Identity-aware routing

Site-to-site

Mesh networking

Edge networking

Kubernetes integration

Final Refined Recommendation

If I were designing this from scratch today, I would build:

Control Plane | Go | Gin | ConnectRPC | PostgreSQL |
Redis | NATS JetStream | Podman

Networking | WireGuard | AmneziaWG | Hysteria2 | QUIC-
first architecture

Shared Core | Rust

Client Apps | Flutter | Android | iOS | HarmonyOS |
Windows | Linux | macOS | Web

Infrastructure | OpenTelemetry | Prometheus | Grafana

Security | Passkeys | OIDC | Zero Trust

Features | Multi-network exposure | Network connectors |
Mullvad-class UX | Relay mesh | Obfuscation | QUIC |
Multi-hop | Split tunneling | Kill switch | DNS protection |
Real-time event system

This architecture is substantially stronger, more scalable, and more future-proof than a pure WireGuard/Hysteria deployment and positions the product closer to a commercial platform like Mullvad, Tailscale, NetBird, and Cloudflare Zero Trust combined.

Let's continue at the level of designing something that could realistically compete with Mullvad, NetBird, Tailscale, Nebula, ZeroTier, and enterprise ZTNA platforms in 2026-2030.

The Biggest Architectural Decision

Most VPN products have:

User ↓ VPN Gateway ↓ Network

Your platform should instead become:

User ↓ Identity ↓ Policy ↓ Connectivity ↓ Network Resources

Connectivity becomes a service.

The actual tunnel protocol becomes almost irrelevant.

This is exactly where the market is moving.

Three Plane Architecture

You should separate everything into:

Control Plane Data Plane Management Plane

Control Plane

Responsible for:

Authentication Authorization Device Registration Route
Distribution Network Discovery Relay Selection Policy
Distribution Session Management

Never carries user traffic.

Only metadata.

Data Plane

Responsible for:

WireGuard Hysteria2 QUIC Relays Packet Routing Traffic
Forwarding

Never talks directly to PostgreSQL.

Never performs authorization logic.

Pure packet transport.

Management Plane

Responsible for:

Admin Portal Analytics Billing Auditing Reporting Monitoring

Can be scaled independently.

Connector Architecture

The network-side agent is actually more important than the user
VPN app.

I would call it:

Helix Connector

Connector responsibilities:

Network Discovery Route Advertisement DNS Discovery Tunnel
Management Policy Enforcement Local Firewall Integration
Health Monitoring

Example:

Home Connector

Detected: 192.168.1.0/24 192.168.2.0/24

Advertised: 192.168.1.0/24

Admin approves.

Route becomes available globally.

Dynamic Route Distribution

One of the strongest features.

Traditional VPN:

Static Routes

Your platform:

Dynamic Route Control

Example:

Connector joins:

192.168.10.0/24

Event emitted:

RouteAdvertised

Control plane validates.

Clients instantly receive:

```
{ "route": "192.168.10.0/24", "network": "home" }
```

No restart required.

No reconnect required.

Service Discovery Layer

This is where things get exciting.

Instead of:

192.168.1.15

Users see:

nas.home grafana.lab k8s.cluster git.office

Implement internal DNS.

Similar to:

Tailscale MagicDNS

Cloudflare private DNS

Internal DNS System

Dedicated service:

HelixDNS

Stack:

CoreDNS Go plugins Redis cache

Entity: CoreDNS

Capabilities:

Split DNS Private DNS Service Discovery DNS over HTTPS DNS over QUIC DNSSEC

Device Identity

Do not trust IPs.

Trust devices.

Every device receives:

Device ID Device Certificate Device Public Key

Example:

```
{ "device_id": "a1b2c3", "name": "Milos MacBook",  
  "platform": "macos", "owner": "user123" }
```

Every policy references device identity.

Not IP addresses.

Zero Trust Evolution

Instead of:

Allow subnet

Use:

Allow user AND Allow device AND Allow network AND Allow resource

Example:

Allow:

Developer Group

To Access:

grafana.lab

Only From:

Managed Devices

Application Access

Huge future feature.

Instead of:

Access entire subnet

Expose:

Single Application

Example:

<https://grafana.lab>

without exposing:

192.168.10.0/24

Now you're entering:

Cloudflare Access territory

Google BeyondCorp territory

Relay Architecture Deep Dive

You absolutely need relays.

Never assume:

NAT Traversal Works

Reality:

CGNAT Carrier NAT Hotel WiFi Corporate Firewalls
will break P2P.

Relay Types

Regional Relays

Frankfurt Amsterdam Stockholm Singapore Tokyo New York
Chicago

Dedicated Relays

Customer-owned.

Example:

Customer Deploys:

helix-relay

inside their VPS.

Relay Selection Engine

Clients constantly measure:

Latency Jitter Packet Loss Bandwidth

Algorithm selects:

Best Relay

automatically.

QUIC-First Future

I strongly recommend:

Everything QUIC

Not just VPN traffic.

Control plane too.

Why

QUIC gives:

Connection Migration Multiplexing Encryption NAT Resilience
Mobile Roaming

Example:

User switches:

WiFi ↓ 5G

Connection survives.

No reconnect.

Huge UX advantage.

Session Layer

Every device maintains:

Persistent QUIC Session

to control plane.

Contains:

```
{ "device":"abc", "version":"2.0", "capabilities":[] }
```

Server pushes events instantly.

Multi-Hop Architecture

Mullvad-like feature.

Example:

User ↓ Norway ↓ Sweden ↓ Home Network

Or:

User ↓ Germany ↓ Netherlands ↓ Office

Build relay chains dynamically.

Obfuscation Framework

Do NOT tie obfuscation to one protocol.

Create:

```
type ObfuscationProvider interface { Wrap() Unwrap() }
```

Implementations:

Salamander Amnezia HTTP3 Mimic TLS Mimic Future DPI Bypass

Mobile Architecture

This becomes critical.

Android and iOS aggressively kill background apps.

Architecture:

UI Layer ↓ Native Service ↓ Rust Core ↓ Tunnel Engine

UI never owns tunnel state.

Native service owns tunnel state.

This dramatically improves reliability.

Rust Core Deep Dive

I would move almost everything reusable into Rust.

Modules:

helix-core helix-routing helix-policy helix-events helix-quic helix-crypto helix-dns helix-device

Shared across:

Android iOS Windows Linux macOS HarmonyOS Aurora
WebAssembly

Web Client

Most VPN projects fail here.

You should support:

Browser Access Mode

without VPN.

Example:

User opens:

portal.company.com

and accesses:

grafana.lab

through browser.

No VPN installation required.

Huge enterprise feature.

Kubernetes Native Support

This should be a first-class citizen.

Deploy:

Helix Connector Operator

inside Kubernetes.

Capabilities:

Expose Service Expose Namespace Expose Cluster

Example:

apiVersion: helix.io/v1 kind: ExposedService

spec: service: grafana namespace: monitoring

Instantly accessible globally.

Observability Architecture

Everything observable.

Metrics:

Connected Devices Relay Usage Network Usage Route Count DNS
Queries Latency Packet Loss Handshake Time

Use:

Prometheus

Grafana

OpenTelemetry

Scalability Targets

Design for:

100,000+ devices 10,000+ networks 1,000+ relays Millions of routes

From day one.

Meaning:

PostgreSQL

Only:

Source of Truth

Redis

Only:

Hot Cache

NATS

Only:

Event Bus

Never bypass these boundaries.

Ultimate v1 Product Definition

The strongest v1 would be:

Helix Platform

Control Plane ✓ Go ✓ Gin ✓ ConnectRPC ✓ PostgreSQL ✓ Redis ✓ NATS

Protocols ✓ WireGuard ✓ AmneziaWG ✓ Hysteria2

Features ✓ Multi-network access ✓ Connector agents ✓ Dynamic routing ✓ Service discovery ✓ Internal DNS ✓ QUIC transport ✓ Obfuscation ✓ Kill switch ✓ Split tunneling ✓ Multi-hop ✓ Relay mesh

Clients ✓ Android ✓ iOS ✓ Windows ✓ Linux ✓ macOS ✓ HarmonyOS ✓ Aurora ✓ Web

Shared Runtime ✓ Rust

Security ✓ OIDC ✓ Passkeys ✓ Zero Trust

Infrastructure ✓ Podman ✓ OpenTelemetry ✓ Prometheus ✓ Grafana

At that point you're no longer building a VPN product. You're building a modern secure connectivity platform that can serve homes, homelabs, MSPs, enterprises, Kubernetes environments, and edge networks from the same architecture.

Now we reach the level where most VPN startups completely fail:

Distributed Systems Architecture, Protocol Design, Control Plane Scaling, Edge Networking, Platform SDK, and eventually becoming something closer to a networking operating system than a VPN service.

The Real Product

Your actual product is NOT:

VPN

Your actual product is:

Connectivity Operating System

Think:

AWS VPC + Tailscale + Mullvad + Cloudflare Zero Trust +
Kubernetes Networking

inside one platform.

Core Internal Architecture

I would split the entire platform into micro-platforms.

Helix Platform

|— Identity |— Device |— Network |— Connector |— Relay |—
Routing |— Policy |— DNS |— Telemetry |— Billing |—
Notifications |— Audit |— Protocol |— SDK

Each service owns exactly one domain.

Device Service

Everything revolves around devices.

Every device gets:

```
{ "device_id":"uuid", "public_key":"...", "owner":"...",  
  "platform":"...", "version":"...", "capabilities":[] }
```

Device lifecycle:

Registered Approved Connected Suspended Revoked Deleted

Every transition is event sourced.

Event Sourcing

I would strongly recommend:

PostgreSQL + NATS JetStream

Event examples:

DeviceCreated DeviceConnected DeviceDisconnected
NetworkCreated NetworkDeleted RouteAdvertised PolicyApplied
RelaySelected

Nothing modifies state directly.

Everything emits events.

CQRS Architecture

For this platform:

Command Side

CreateDevice CreateNetwork JoinNetwork ApproveDevice

↓

Events

↓

Read Side

Optimized Queries

This becomes extremely important when you hit:

100k+ devices

Network Objects

Most VPNs treat networks as subnets.

Huge mistake.

Your platform should define:

```
{ "id":"network-1", "name":"Office", "type":"private", "routes":[] }
```

Network becomes a first-class object.

Network Types

Support:

Private Network Public Network Connector Network Site Network
Application Network Mesh Network

Site-to-Site

One of the strongest future features.

Example:

Office A ↓ Helix ↓ Office B

No user client involved.

Just connectors.

Connector Clustering

Huge enterprise feature.

Instead of:

1 Connector

Support:

Connector Cluster

connector-1 connector-2 connector-3

Active-active.

Connector Election

Use:

Raft

or

etcd

for cluster state.

Entity references:

etcd

Route Intelligence Engine

This becomes one of your most valuable components.

Example:

Network A

10.0.0.0/16

Network B

10.0.0.0/16

Conflict.

Platform detects automatically.

Suggests:

NAT Translation Virtual Prefix

Overlay Address Space

Create your own address space.

Example:

100.64.0.0/10

(similar to Tailscale)

Every device receives:

100.64.x.x

stable forever.

Virtual Network Layer

Eventually:

Home Office Lab Cloud

become virtualized.

User doesn't care about physical routes anymore.

Only resources.

Resource Model

Instead of exposing:

192.168.1.20

Expose:

nas.home

Instead of:

10.20.1.5

Expose:

git.office

Resources become first-class entities.

Service Mesh Integration

Future enterprise feature.

Integrate:

Istio

Linkerd

Allow:

Kubernetes Service ↓ Helix ↓ Remote User

without ingress exposure.

Application Connector

One revolutionary feature.

Traditional VPN:

Expose Subnet

Helix:

Expose Application

Connector runs:

applications:

- name: grafana url: <http://10.0.0.5:3000>

- name: gitlab url: <http://10.0.0.8>

Platform automatically creates:

grafana.company gitlab.company

Identity-Aware Proxy

Eventually add:

Helix Access

Equivalent to:

Cloudflare Access

Google BeyondCorp

Request flow:

User ↓ Identity ↓ Policy ↓ Application

Relay Mesh Network

This becomes one of the most technically interesting parts.

Not:

One VPS

Instead:

Global Relay Mesh

Frankfurt Amsterdam Oslo Stockholm London New York Chicago
Dallas Singapore Tokyo Sydney

Each relay runs:

helix-relay

Relay Coordination

Control plane distributes:

Load Latency Congestion Capacity

in real time.

QUIC Mesh

This is where I would innovate.

Relays communicate via:

QUIC

not TCP.

Benefits:

Migration Multiplexing Encryption Low Latency

Entire backbone becomes QUIC-native.

Multi-Hop Engine

Most VPNs hardcode this.

Don't.

Create:

```
{ "entry":"oslo", "middle":"stockholm", "exit":"frankfurt" }
```

Generated dynamically.

Smart Path Selection

Future AI-assisted routing.

Inputs:

Latency Packet Loss Congestion Cost Location

Outputs:

Best Path

Every few seconds.

DNS Architecture

Build a dedicated subsystem.

HelixDNS

Based on:

CoreDNS

Features:

Private Zones Magic DNS Split DNS DoH DoQ DNSSEC

DoQ (DNS-over-QUIC) should be default.

Protocol Abstraction Layer

One of the most important components.

```
type Transport interface { Connect() Disconnect() Send()  
Receive() }
```

Implementations:

WireGuard Hysteria2 AmneziaWG Future QUIC Transport Future
MASQUE Future WebTransport

MASQUE Future

Watch this carefully.

Entity: MASQUE

MASQUE may become extremely important.

Benefits:

HTTP/3 Native QUIC Native Proxy Native Firewall Friendly

Potentially future replacement for parts of today's VPN stack.

Client Runtime

I would design:

Helix Runtime

Written entirely in Rust.

Modules:

helix-core helix-routing helix-dns helix-policy helix-device helix-
events helix-transport helix-obfuscation helix-relay

Flutter Integration

Flutter should only be:

Presentation Layer

Nothing else.

Rust owns:

Networking Crypto Routing Tunnel Policies State

Flutter owns:

UI Settings Administration

This dramatically improves reliability.

Web Platform

Do not treat web as secondary.

Build:

Helix Web

Capabilities:

Admin Monitoring Resource Access Application Access Network
Management Device Management

Enterprise Features (Future)

SCIM

Automatic user provisioning.

Entity: SCIM

SAML

Enterprise SSO.

Entity: SAML

OIDC

Modern auth.

Entity: OpenID Connect

Long-Term Vision (2030)

The final evolution isn't:

VPN Service

It's:

Global Connectivity Fabric

Architecture:

Users Devices Applications Clusters Offices Datacenters Clouds
IoT Edge Nodes

all connected through:

Identity + Policy + Connectivity

with transport dynamically choosing:

WireGuard AmneziaWG Hysteria2 QUIC MASQUE Future
protocols

underneath.

At that stage, the platform starts resembling a self-hostable
alternative to a combination of Cloudflare Zero Trust, Tailscale,
NetBird, ZeroTier, and parts of modern SD-WAN systems rather
than a traditional VPN product.

Now we enter the territory where you can create something that
existing VPN vendors simply cannot easily replicate because their
architectures are already constrained by legacy decisions.

The next step is designing the platform as a distributed
connectivity operating system with a programmable networking
core.

HelixOS Architecture

Think of the entire platform as:

HelixOS

├─ Connectivity Kernel ├─ Identity Kernel ├─ Policy Kernel ├
Routing Kernel ├─ Service Discovery Kernel ├─ Resource Kernel
├─ Transport Kernel ├─ Security Kernel ├─ Automation Kernel ├
Edge Kernel

Every component communicates through events.

Connectivity Kernel

This becomes the heart of the platform.

Current VPNs:

Connect() Disconnect()

Helix:

Discover Authorize Negotiate Optimize Connect Monitor Adapt
Heal

Connection lifecycle never ends.

It continuously optimizes.

Self-Healing Network

A huge differentiator.

Traditional VPN:

Connection Lost ↓ Reconnect

Helix:

Packet Loss Detected ↓ Path Quality Degrading ↓ Alternative
Path Found ↓ Traffic Migrated

No user interaction.

No disconnect.

No visible outage.

Connectivity Graph

Every component becomes a graph node.

User Device Connector Relay Resource Network Application
Cluster

Represented internally:

Node Edge Relationship Capability

This graph becomes one of the most valuable assets.

Resource-Centric Networking

Current VPNs expose:

IPs Subnets Routes

Users hate this.

Expose:

Resources

Examples:

Office NAS Lab Grafana Development Cluster Production API
Home Printer

Never show:

10.20.30.0/24

unless advanced mode.

Resource Registry

New service:

helix-resource

Stores:

```
{ "resource": "grafana", "network": "lab", "owner": "ops-team",  
  "access": "policy-id" }
```

Service Catalog

Enterprise users will love this.

User opens client:

Resources

✓ Grafana ✓ GitLab ✓ Jenkins ✓ NAS ✓ Kubernetes ✓ PostgreSQL

Instant access.

No IP knowledge required.

Entity references:

Grafana

GitLab

Jenkins

PostgreSQL

Connector Evolution

The connector should evolve into a miniature edge platform.

Not merely:

VPN Agent

Instead:

Edge Runtime

Capabilities:

Routing DNS Firewall Application Publishing Service Discovery
Policy Enforcement Monitoring Caching

Edge Runtime

I would actually design:

helix-edge

similar to:

Kubernetes node

Edge gateway

SD-WAN appliance

Written in Rust + Go.

Edge Plugins

One of the most important future features.

Plugin architecture:

```
trait Plugin { fn initialize(); fn start(); fn stop(); }
```

Examples:

Docker Plugin Podman Plugin Kubernetes Plugin DNS Plugin
Firewall Plugin Storage Plugin MQTT Plugin

Entity references:

Docker

Podman

MQTT

Universal Edge Nodes

Imagine:

Raspberry Pi VPS Mini PC Cloud VM NAS Router

all running:

helix-edge

and becoming first-class platform members.

Edge Marketplace

Much later.

Connectors could deploy:

Monitoring DNS Proxies Applications Automation

through signed packages.

Like:

Helix Package Registry

Control Plane Evolution

Current design:

Go Gin Postgres Redis NATS

Still correct.

But eventually split.

identity-service device-service network-service policy-service
resource-service relay-service dns-service billing-service audit-
service

Each independently scalable.

Internal Communication

Avoid REST internally.

Use:

ConnectRPC gRPC Protobuf

Everywhere.

Event Backbone

I would make NATS absolutely central.

Entity:

NATS

Topics:

device.* network.* connector.* relay.* policy.* resource.* dns.*
audit.* billing.*

State Synchronization

One extremely difficult problem:

100,000+ devices

must receive updates instantly.

Solution:

Delta Synchronization

Not full state.

Only changes.

Example:

```
{ "type": "route_added", "route": "10.0.0.0/24" }
```

instead of sending everything.

Global Configuration Engine

Inspired by Kubernetes.

Everything becomes declarative.

Example:

apiVersion: helix.io/v1

kind: Network

metadata: name: office

spec: routes: - 10.10.0.0/16

Infrastructure-as-Code

This becomes huge.

Users can define:

Users Groups Networks Policies Resources Relays Connectors
through Git.

GitOps networking.

SDK Platform

A massively underestimated opportunity.

Create:

Helix SDK

Languages:

Go Rust TypeScript Kotlin Swift Python

Capabilities:

Device Management Network Management Resource
Management Policies Automation Events

Embedded Integrations

Future integrations:

Home Assistant

Entity:

Home Assistant

Kubernetes

Entity:

Kubernetes

Terraform

Entity:

Terraform

Ansible

Entity:

Ansible

Connectivity AI Layer

Not AI hype.

Actually useful.

Analyze:

Latency Packet Loss Congestion Failures Usage Patterns

Recommend:

Better Relay Better Route Better Protocol Better Exit Path

Automatically.

Multi-Protocol Future

Today:

WireGuard AmneziaWG Hysteria2

Tomorrow:

MASQUE WebTransport HTTP/3 Tunnels Custom QUIC Overlay

Architecture must support protocol hot-swapping.

Browser-Native Connectivity

One of the most disruptive future directions.

Use:

WebTransport WebRTC QUIC

to provide access directly from browsers.

No client installation.

No extensions.

No VPN.

Just:

login authorize connect

WebAssembly Strategy

The Rust core should compile to:

Native WASM

Then:

Desktop Mobile Web Edge

all share the same runtime logic.

This is arguably the most important long-term architectural decision.

Unified Client Architecture

I would eventually structure all clients like:

Flutter UI ↓ Platform Adapter ↓ Rust Core ↓ Transport Layer ↓
Kernel Network APIs

Platforms:

Android iOS HarmonyOS Aurora Windows Linux macOS Web

One connectivity engine.

One policy engine.

One routing engine.

One state engine.

One event engine.

The Endgame

The final system is not:

VPN-as-a-Service

It becomes:

Connectivity-as-a-Platform

where a user can connect:

Devices Networks Applications Kubernetes Clusters Databases
IoT Systems Edge Locations Offices Clouds

through a unified identity, policy, routing, and transport fabric that happens to use WireGuard, QUIC, Hysteria2, MASQUE, and future transports underneath.

That is the architecture that has the potential to become a genuinely category-defining platform rather than another VPN management product.